SYNTH-06: Synthetic Analytics & Alerting

> **Series:** SYNTH — Synthetic Monitoring | **Notebook:** 6 of 6 | **Created:** December 2025 | **Last Updated:** 10/02/2026

Dashboards, SLOs, and Alerting Strategies

This notebook covers advanced analytics for synthetic monitoring, including building dashboards, configuring SLOs, and implementing effective alerting strategies using the latest Dynatrace platform.

Table of Contents

1. Analytics Overview
2. Availability Analysis
3. Performance Analysis
4. Location Comparison
5. SLO Configuration
6. Alerting Strategies
7. Dashboard Building
8. Series Complete!

Prerequisites

- ✅ Access to a Dynatrace environment with Synthetic Monitoring
- ✅ Completed SYNTH-01 through SYNTH-05
- ✅ Active synthetic monitors generating data

> **Query convention:** These analyses span all monitor types, so they read monitor-level execution events with `filter endsWith(event.type, "_monitor_execution")` and group by `monitor.name`. Success is `result.state == "SUCCESS"`; duration is `result.statistics.duration / 1ms`.

1. Analytics Overview

Key Metrics for Synthetic Monitoring

Metric Category	Metrics	Use Case
Availability	Success rate, failure count	SLA compliance
Performance	Response time, TTFB, DNS	User experience
Reliability	Consecutive failures, MTTR	Stability
Geographic	Per-location metrics	Regional issues

Data Sources

Source	Description	Best For
`dt.synthetic.events`	Execution results (events)	Per-execution analysis, failures, percentiles
`dt.synthetic.detailed_events`	Request-level detail for HTTP steps (`http_step_execution_details`)	Deep-dive on individual HTTP requests
`timeseries dt.synthetic.*`	Pre-aggregated metrics	Long-term trends, dashboards
`dt.entity.synthetic_test` / `dt.entity.http_check` / `dt.entity.multiprotocol_monitor`	Monitor definitions (classic — deprecated in DQL)	Configuration, name resolution
`dt.entity.synthetic_location`	Location info (classic — deprecated in DQL)	Geographic analysis

Source	Description	Best For
`smartscapeNodes "BROWSER_MONITOR"` / `"HTTP_MONITOR"` / `"NETWORK_AVAILABILITY_MONITOR"`	Monitor definitions (Smartscape — **preferred**)	Configuration; `name` is already the display name, so no `entityName()` step
`smartscapeNodes "BROWSER_MONITOR_STEP"` / `"HTTP_MONITOR_STEP"`	Step definitions as **separate nodes**	Per-step configuration; `traverse` the `belongs_to` edge to reach the parent monitor
`smartscapeNodes "SYNTHETIC_LOCATION"`	Location info (Smartscape — **preferred**)	Geographic analysis with `location_type`, `stage`, `cloud.provider`, `geo.*` — none of which exist on the classic entity

> **Note on the entity rows.** SaaS 1.344 (released 07/27/2026, **staged tenant rollout from 07/29/2026**) makes `dt.smartscape.*` the primary entity surface for the Synthetic app. The classic tables above still return rows, but DQL queries against classic synthetic entities are documented as deprecated — *"They remain supported for as long as Dynatrace Classic is supported, so you can adopt the new model at your own pace."* Keep them as a fallback, not a target. Two mapping traps: the network-monitor node is `NETWORK_AVAILABILITY_MONITOR` (**not** `MULTIPROTOCOL_MONITOR`), and classic `synthetic_test` splits into `BROWSER_MONITOR` + `BROWSER_MONITOR_STEP`. See SYNTH-01 § 4 for the full mapping table and FAQ-16 for migration mechanics.
>
> **Sources:** Synthetic monitors in Smartscape (DT docs).

2. Availability Analysis

Calculating Availability

```
Availability = (Successful Executions / Total Executions) × 100%

Example SLA Targets:
- 99.9% = max 8.76 hours downtime/year
- 99.5% = max 43.8 hours downtime/year
- 99.0% = max 87.6 hours downtime/year
```

> **Count each scheduled run once.** A browser monitor that retries on error writes the failed attempt *and* a re-run (`execution.type` `RE_RUN`) for the same scheduled run. The Synthetic events model describes `execution.retry_on_error` as *"Should we ignore this execution attempt (which was a failure) - will there be retry."* Counting both inflates failures: on the validation tenant, 2,015 of 4,701 browser executions in 24 hours were re-runs. The aggregating queries below drop the superseded attempt with `filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")` . The `coalesce` matters: HTTP executions carry no value for the field, and without it their failures would be filtered out as well.
>
> **Monitor names are not unique.** A browser monitor and an HTTP monitor can share a name, so per-monitor queries group by `dt.synthetic.monitor.id` as well as `monitor.name` .
>
> **Sources:** Synthetic events model (DT docs) — *"execution.type MUST be one of the following"* (the list includes `re-run` and `standard` ; the tenant returned `RE_RUN` / `STANDARD` for browser and `standard` for HTTP, read 10/02/2026).

```
// Overall synthetic availability (last 7 days)
// Execution-weighted across every monitor and location. For a downtime estimate,
// use the per-monitor query below -- a tenant-wide failed count cannot be turned
// into minutes, because it sums failures across monitors, locations and
frequencies.
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| summarize {
    total_executions = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
  }
| fieldsAdd availability_pct = round((successful * 100.0) / total_executions,
decimals: 3)
```

```
// Estimated failed minutes per monitor (last 7 days)
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test on
synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* —
Synthetic events model, 09/21/2026.
// Share of executions that failed x the minutes in the window. This needs no
// frequency assumption and never exceeds the window, unlike failed-count x 5,
// which sums failures across every monitor and location.
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| summarize {
    executions = count(),
    failed = countIf(result.state == "FAIL"),
    locations = countDistinct(dt.smartscape.synthetic_location)
  }, by: {monitor.name, dt.synthetic.monitor.id}
| fieldsAdd failed_pct = round(failed * 100.0 / executions, decimals: 2)
| fieldsAdd est_failed_minutes = round(failed * 10080.0 / executions, decimals: 0)
// 10,080 = minutes in 7 days
| sort est_failed_minutes desc
| limit 20
```

```
// Availability by monitor (last 7 days)
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| summarize {
    total = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
  }, by: {monitor.name, dt.synthetic.monitor.id}
| fieldsAdd availability_pct = round((successful * 100.0) / total, decimals: 3)
| sort availability_pct asc
| limit 30
```

```
// Availability trend over time
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| fieldsAdd hour_bucket = bin(timestamp, 1h)
| summarize {
    success_count = countIf(result.state == "SUCCESS"),
    total_count = count()
  }, by: {hour_bucket}
| fieldsAdd availability_pct = round((success_count * 100.0) / total_count,
decimals: 2)
| sort hour_bucket desc
```

```
// Daily availability report
fetch dt.synthetic.events, from: now() - 30d
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| fieldsAdd day = formatTimestamp(timestamp, format: "yyyy-MM-dd")
| summarize {
    total = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
  }, by: {day}
| fieldsAdd availability_pct = round((successful * 100.0) / total, decimals: 3)
| sort day desc
| limit 30
```

3. Performance Analysis

Performance Metrics

Starting points from community practice — set yours from each monitor's observed baseline.

Metric	Description	Good	Warning	Critical
Response Time	Total execution time	< 2s	2-5s	> 5s
DNS	DNS resolution	< 50ms	50-200ms	> 200ms
Connect	TCP connection	< 100ms	100-300ms	> 300ms
TTFB	Time to first byte	< 500ms	500ms-1s	> 1s

```
// Response time percentiles by monitor
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "SUCCESS"
| summarize {
    p50 = percentile(result.statistics.duration, 50),
    p75 = percentile(result.statistics.duration, 75),
    p90 = percentile(result.statistics.duration, 90),
    p95 = percentile(result.statistics.duration, 95),
    p99 = percentile(result.statistics.duration, 99),
    executions = count()
  }, by: {monitor.name, dt.synthetic.monitor.id}
| fieldsAdd p50_ms = round(p50/1ms, decimals:1), p75_ms = round(p75/1ms,
decimals:1),
            p90_ms = round(p90/1ms, decimals:1), p95_ms = round(p95/1ms,
decimals:1),
            p99_ms = round(p99/1ms, decimals:1)
| fieldsRemove p50, p75, p90, p95, p99
| sort p95_ms desc
| limit 20
```

```
// Performance trend comparison (this week vs last week)
fetch dt.synthetic.events, from: now() - 14d
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "SUCCESS"
| fieldsAdd week = if(timestamp &gt; now() - 7d, "This Week", else: "Last Week")
| summarize {
    avg_response_ms = round(avg(result.statistics.duration / 1ms), decimals: 1),
    p95 = percentile(result.statistics.duration / 1ms, 95),
    executions = count()
  }, by: {monitor.name, dt.synthetic.monitor.id, week}
| fieldsAdd p95_response_ms = round(p95, decimals: 1)
| fieldsRemove p95
| sort monitor.name asc, week asc
```

```
// Timing breakdown analysis (HTTP step records)
// TTFB already includes DNS + TCP + TLS, so these phases overlap — do not sum
them.
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_step_execution"
| filter result.status.code == 0
| summarize {
    avg_dns_ms     = round(avg(result.statistics.host_name_resolution_time) / 1ms,
decimals: 2),
    avg_connect_ms = round(avg(result.statistics.tcp_connect_time) / 1ms,
decimals: 2),
    avg_tls_ms     = round(avg(result.statistics.tls_handshake_time) / 1ms,
decimals: 2),
    avg_ttfb_ms    = round(avg(result.statistics.time_to_first_byte) / 1ms,
decimals: 2),
    avg_total_ms   = round(avg(result.statistics.duration) / 1ms, decimals: 2)
  }, by: {monitor.name}
| sort avg_total_ms desc
| limit 20
```

```
// Performance anomalies (max response time &gt; 2x average per monitor)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "SUCCESS"
| fieldsAdd response_ms = result.statistics.duration / 1ms
| summarize {
    avg_response = avg(response_ms),
    max_response = max(response_ms),
    min_response = min(response_ms),
    executions = count()
  }, by: {monitor.name, dt.synthetic.monitor.id}
| fieldsAdd deviation_factor = round(max_response / avg_response, decimals: 1)
| filter deviation_factor &gt; 2
| sort deviation_factor desc
| limit 50
```

4. Location Comparison

Geographic Analysis

Compare synthetic results across locations to:
- Identify regional performance issues
- Detect CDN or DNS problems
- Validate global availability
- Optimize user experience by region

```
// Availability by location
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test on
synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* —
Synthetic events model, 09/21/2026.
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| summarize {
    total = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
  }, by: {dt.smartscape.synthetic_location}
| fieldsAdd availability_pct = round((successful * 100.0) / total, decimals: 2)
| fieldsAdd location = getNodeName(dt.smartscape.synthetic_location)
| sort availability_pct asc
| limit 30
```

```
// Performance by location
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test on
synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* —
Synthetic events model, 09/21/2026.
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "SUCCESS"
| summarize {
    avg_response_ms = round(avg(result.statistics.duration / 1ms), decimals: 1),
    p50 = percentile(result.statistics.duration, 50),
    p95 = percentile(result.statistics.duration, 95),
    executions = count()
  }, by: {dt.smartscape.synthetic_location}
| fieldsAdd p50_ms = round(p50 / 1ms, decimals: 1), p95_ms = round(p95 / 1ms,
decimals: 1)
| fieldsRemove p50, p95
| fieldsAdd location = getNodeName(dt.smartscape.synthetic_location)
| sort avg_response_ms desc
| limit 30
```

```
// Location performance heatmap (monitor x location)
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test on
synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* —
Synthetic events model, 09/21/2026.
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| summarize {
    executions = count(),
    availability_pct = round(countIf(result.state == "SUCCESS") * 100.0 / count(),
decimals: 2),
    avg_response_ms = round(avg(result.statistics.duration / 1ms), decimals: 1)
  }, by: {monitor.name, dt.synthetic.monitor.id, dt.smartscape.synthetic_location}
| fieldsAdd location = getNodeName(dt.smartscape.synthetic_location)
| sort monitor.name asc, avg_response_ms desc
```

```
// Location-specific failures
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test on
synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* —
Synthetic events model, 09/21/2026.
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "FAIL"
| summarize {
    failure_count = count(),
    unique_monitors = countDistinct(dt.synthetic.monitor.id),
    error_types = collectDistinct(result.status.message)
  }, by: {dt.smartscape.synthetic_location}
| fieldsAdd location = getNodeName(dt.smartscape.synthetic_location)
| sort failure_count desc
| limit 20
```

5. SLO Configuration

Service Level Objectives for Synthetic

SLO Type	Metric	Example Target
Availability	Success rate	99.9%

SLO Type	Metric	Example Target
Performance	Response time P95	< 3000ms
Error Budget	Allowed failures	0.1% of executions

Creating Synthetic SLOs

Create SLOs in the **Service-Level Objectives** app on a DQL query — the availability and P95 queries below are the shape it takes. The SLO series covers defining the SLI, error budgets and burn-rate alerting (SLO-02 to SLO-04).

```
// SLO calculation - Availability (30-day window)
fetch dt.synthetic.events, from: now() - 30d
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| summarize {
    total_executions = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
  }, by: {monitor.name, dt.synthetic.monitor.id}
| fieldsAdd sli_availability = round((successful * 100.0) / total_executions,
decimals: 4)
| fieldsAdd slo_target = 99.9
| fieldsAdd error_budget_total = total_executions * 0.001  // 0.1% budget, in
failed runs (not rounded before use)
| fieldsAdd error_budget_remaining = error_budget_total - failed
| fieldsAdd error_budget_pct = if(error_budget_total &gt; 0,
    round((error_budget_remaining * 100.0) / error_budget_total, decimals: 1))
| fieldsAdd slo_status = if(sli_availability &gt;= slo_target, "✅ Met", else: "❌
Breached")
| sort sli_availability asc
| limit 30
```

```
// SLO calculation - Performance (P95 &lt; 3000ms)
fetch dt.synthetic.events, from: now() - 30d
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "SUCCESS"
| fieldsAdd response_ms = result.statistics.duration / 1ms
| summarize {
    executions = count(),
    p95 = percentile(response_ms, 95),
    within_target = countIf(response_ms &lt; 3000)
  }, by: {monitor.name, dt.synthetic.monitor.id}
| fieldsAdd p95_response_ms = round(p95, decimals: 1)
| fieldsRemove p95
| fieldsAdd sli_performance = round((within_target * 100.0) / executions,
decimals: 2)
| fieldsAdd slo_target = 95.0  // 95% of requests &lt; 3s
| fieldsAdd slo_status = if(sli_performance &gt;= slo_target, "✅ Met", else: "❌
Breached")
| sort sli_performance asc
| limit 30
```

```
// Error budget burn rate (last 7 days)
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| fieldsAdd day = formatTimestamp(timestamp, format: "yyyy-MM-dd")
| summarize {
    total = count(),
    failed = countIf(result.state == "FAIL")
  }, by: {monitor.name, dt.synthetic.monitor.id, day}
| fieldsAdd daily_error_rate = round((failed * 100.0) / total, decimals: 3)
| fieldsAdd budget_consumed = round(failed * 100.0 / (total * 0.001), decimals: 1)
// vs 0.1% budget
| sort monitor.name asc, day desc
```

6. Alerting Strategies

Alert Types

Alert Type	Trigger	Use Case
Availability	Consecutive failures	Outage detection

Alert Type	Trigger	Use Case
Performance	Response time threshold	Degradation
SLO	Error budget consumption	Proactive warning
SSL	Certificate expiration	Security

Alert Configuration Best Practices

Setting	Recommendation	Reason
Consecutive failures	2-3	Avoid false positives
Location threshold	2+ locations	Confirm not local issue
Automatic retry on error (browser monitors)	Enabled	Absorbs a single transient failure before the execution counts as failed

Synthetic outage handling has no separate alert-delay or auto-resolve-after-N-successes setting. Global and local outages are driven by the consecutive-failure count and the location threshold above; performance problems close once *"no performance threshold is violated in the five most recent executions"*. Browser monitors can also retry once: the option *"configures the monitor to avoid false positives by making one more execution after the first one failed"*.

> **Sources:** Synthetic alerting overview (DT docs) — *"You can specify how many consecutive executions must fail so the problem is generated"*.

```
// Monitors with recent failures (potential outages)
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test on
synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* —
Synthetic events model, 09/21/2026.
fetch dt.synthetic.events, from: now() - 1h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "FAIL"
| summarize {
    failure_count = count(),
    last_failure = max(timestamp),
    first_failure = min(timestamp)
  }, by: {monitor.name, dt.synthetic.monitor.id, dt.smartscape.synthetic_location}
| filter failure_count &gt;= 2
| fieldsAdd location = getNodeName(dt.smartscape.synthetic_location)
| sort failure_count desc
| limit 20
```

```
// Performance degradation alerts (P95 &gt; threshold)
fetch dt.synthetic.events, from: now() - 1h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "SUCCESS"
| summarize {
    avg_response_ms = round(avg(result.statistics.duration / 1ms), decimals: 1),
    p95 = percentile(result.statistics.duration / 1ms, 95),
    executions = count()
  }, by: {monitor.name, dt.synthetic.monitor.id}
| fieldsAdd p95_response_ms = round(p95, decimals: 1)
| fieldsRemove p95
| filter p95_response_ms &gt; 5000  // &gt; 5 seconds threshold
| sort p95_response_ms desc
```

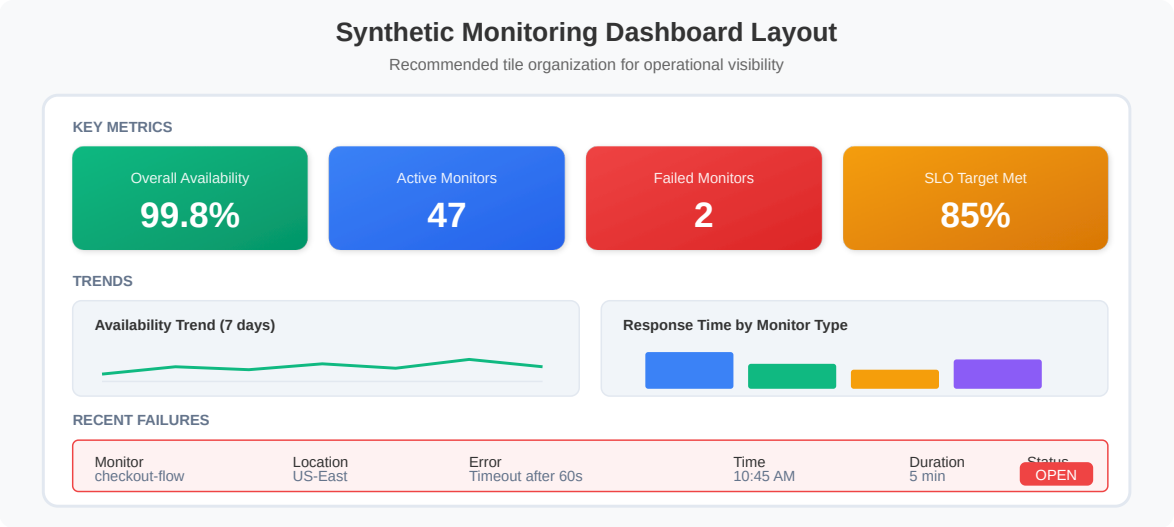
```
// Multi-location failure detection (global outage indicator)
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test on
synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* —
Synthetic events model, 09/21/2026.
fetch dt.synthetic.events, from: now() - 30m
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "FAIL"
| summarize {
    failing_locations = countDistinct(dt.smartscape.synthetic_location),
    failure_count = count(),
    locations = collectDistinct(dt.smartscape.synthetic_location)
  }, by: {monitor.name, dt.synthetic.monitor.id}
| filter failing_locations &gt;= 2
| fieldsAdd severity = if(failing_locations &gt;= 3, "CRITICAL", else: "WARNING")
| sort failing_locations desc
```

7. Dashboard Building

Recommended Dashboard Tiles

Tile Type	Content	Visualization
Overall Health	Availability %	Single value
Availability Trend	Hourly availability	Line chart
Response Time	P95 by monitor	Bar chart
Location Map	Geographic availability	World map
Failures Table	Recent failures	Table
SLO Status	Error budget	Gauge

Dashboard Layout




```
// Dashboard tile: Overall availability (single value)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| summarize {
    availability_pct = round(countIf(result.state == "SUCCESS") * 100.0 / count(),
decimals: 2)
  }
```

```
// Dashboard tile: Healthy vs failing monitors (last hour)
fetch dt.synthetic.events, from: now() - 1h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| summarize {
    failed_recent = countIf(result.state == "FAIL"),
    executions = count()
  }, by: {monitor.name, dt.synthetic.monitor.id}
| summarize {
    total_monitors = count(),
    healthy = countIf(failed_recent == 0),
    failing = countIf(failed_recent &gt; 0)
  }
```

```
// Dashboard tile: Response time by monitor (bar chart)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "SUCCESS"
| summarize {
    p95 = percentile(result.statistics.duration / 1ms, 95)
  }, by: {monitor.name, dt.synthetic.monitor.id}
| fieldsAdd p95_response_ms = round(p95, decimals: 1)
| fieldsRemove p95
| sort p95_response_ms desc
| limit 10
```

```
// Dashboard tile: Recent failures table
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test on
synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* —
Synthetic events model, 09/21/2026.
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| filter result.state == "FAIL"
| fields timestamp,
         monitor = monitor.name,
         location = getNodeName(dt.smartscape.synthetic_location),
         status = result.status.message,
         detail = result.status.details
| sort timestamp desc
| limit 20
```

```
// Dashboard tile: Hourly availability trend (line chart)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")
// one result per scheduled run
| fieldsAdd hour_bucket = bin(timestamp, 1h)
| summarize {
    success_count = countIf(result.state == "SUCCESS"),
    total_executions = count(),
    failures = countIf(result.state == "FAIL")
  }, by: {hour_bucket}
| fieldsAdd availability_pct = round((success_count * 100.0) / total_executions,
decimals: 2)
| sort hour_bucket asc
```

Summary

In this notebook, you learned:

✅ **Availability analysis** - Calculating and trending availability with `result.state`
✅ **Performance analysis** - Percentiles, timing breakdown, anomalies
✅ **Location comparison** - Geographic performance analysis
✅ **SLO configuration** - Service level objectives and error budgets
✅ **Alerting strategies** - Outage detection, performance alerts
✅ **Dashboard building** - Key tiles and layouts

Series Complete!

You have completed the Synthetic Monitoring Best Practices series:

1. ✅ **SYNTH-01**: Fundamentals
2. ✅ **SYNTH-02**: Browser Monitors
3. ✅ **SYNTH-03**: HTTP Monitors
4. ✅ **SYNTH-04**: Private Locations
5. ✅ **SYNTH-05**: Network Monitoring
6. ✅ **SYNTH-06**: Analytics & Alerting

References

- Synthetic on Grail (DT docs)
- Synthetic alerting overview (DT docs)
- Synthetic for Workflows (DT docs)
- Service-level objectives (DT docs)
- Dashboards and notebooks (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.